

Proyecto Final — Propuesta inicial: Puebla Ahora, para todas las personas

Angel Rugerio Jiménez #201720

Axel García Arellano #201251

Diego Navarro Sánchez #202497

Yohualli Ollin Luna Reyes #198670

Interacción persona máquina

Dra. Gabriela A. Rodríguez Ruiz

Octubre 27, 2026

- **Materia:** Interacción Persona Máquina (13009A-O26)
- **Profesora:** Dra. Gabriela A. Rodríguez Ruiz
- **Proyecto:** Interfaces Inteligentes para el Bienestar Social
- **Línea temática:** discapacidad e inclusión, adultos mayores
- **Equipo:** Angel Rugerio Jiménez (201720), Axel García Arellano (201251), Diego Navarro Sánchez (202497) y Yohualli Ollin Luna Reyes (198670)
- **Fecha:** 27 de octubre de 2026

> **La idea en corto**
> Vamos a retomar **Puebla Ahora**, la app que Axel y Angel presentaron en el hackathon *Puebla Turística 360°* (Tech Capital Puebla 2026). La app le dice al turista qué lugares están tranquilos y cuáles están llenos. La idea es rehacer su interfaz en **PySide6 y Qt Designer** para que **cualquier persona la pueda usar sin ayuda**, sin importar su edad, cómo ve los colores o cuánto sabe de tecnología.

---

1. De dónde viene la idea

Puebla Ahora responde a una pregunta: *"¿a dónde voy ahora?"*. En la zona Puebla–Cholula–Atlixco muestra un **semáforo** de 23 lugares turísticos (verde: buen momento; amarillo: con precaución; rojo: mejor después). Si el lugar que buscas está lleno, te recomienda otro parecido y más tranquilo. También tiene un asistente al que le puedes preguntar cosas como "tengo 4 horas y llueve, ¿qué hago?".

En el hackathon la hicimos como página web con **React y Vite**, y el backend con **FastAPI** (lo hizo Axel). La profesora conoció la idea y lo que presentamos, y nos dejó continuar con ella para el proyecto final.

2. El problema

La app funciona bien para alguien que ve bien los colores, lee letra pequeña sin problema y está acostumbrado a buscar en listas largas. Pero un servicio para turistas lo usa **cualquiera**. Al revisar la versión del hackathon encontramos barreras concretas:

- **La letra es pequeña:** mucho texto mide entre 11 y 13 px, y no hay forma de agrandarlo dentro de la app.

- **El verde y el rojo se confunden:** simulamos cómo ve los colores una persona con daltonismo (deuteranopía), y el verde ("buen momento") y el rojo ("mejor después") se ven prácticamente iguales.
- **Hay que hacer scroll para ver las opciones:** de los 23 lugares solo se ven entre 5 y 8 a la vez. Para comparar hay que bajar y recordar lo que ya viste.
- **Algunos botones son pequeños** para alguien con temblor en las manos o que usa la pantalla táctil con dificultad.

El detalle de cada hallazgo está en Extras A.

3. Beneficiarios

- **Adultos mayores:** necesitan letra grande, pocos pasos y botones grandes.
- **Personas con daltonismo:** necesitan distinguir el semáforo sin depender del color.
- **Personas con baja visión:** necesitan texto grande y alto contraste.
- **Personas con poca experiencia con la tecnología:** necesitan ver todas las opciones de una vez, sin buscar.
- **Turistas extranjeros:** la app ya tiene versión en inglés.

También gana la Secretaría de Turismo: si más personas pueden usar la app, llegan más reportes y más turistas se reparten entre los lugares.

> **Por completar antes de entregar**
> Una cifra oficial que dimensione la población (por ejemplo, personas de 60 años o más y personas con discapacidad en Puebla, según el Censo 2020 del INEGI), con su fuente.

4. Objetivos

General: que cualquier turista, sin importar su edad o sus capacidades, pueda encontrar un lugar en buen momento y saber cómo llegar, sin ayuda de nadie.

Específicos:

1. Que la persona pueda **agrandar la letra** (al menos tres tamaños) sin que la interfaz se rompa.
2. Que el semáforo se entienda **sin depender del color**: con formas e íconos, y con un modo para daltonismo.

3. Cambiar la lista con scroll por una **barra lateral que se amplía** y muestra todos los lugares a la vez.
4. Que todo se pueda usar con **botones grandes, con el teclado y en voz alta**.
5. **Comprobar con personas reales** que la nueva versión es más fácil de usar que la del hackathon.

5. Funciones

5.1 Lo que ya hace (viene del hackathon)

- **Inicio:** eliges qué te gusta (historia, comida, naturaleza…), cuánto tiempo tienes y dónde estás.
- **Mapa con semáforo:** todos los lugares con su color y una lista ordenada por cuál conviene más.
- **Ficha del lugar:** por qué tiene ese color, horario, costo, clima y cuánto tardas en llegar.
- **"Mejor ve a…":** si el lugar está lleno, te propone otro parecido y te da el enlace para llegar.
- **"¿Cómo está?":** reportas en un toque si el lugar está vacío, normal o lleno, y el semáforo se actualiza.
- **Asistente:** respondes preguntas con un modelo de inteligencia artificial que solo usa los datos de la app y dice de dónde los sacó.
- **Panel de gobierno:** la Secretaría ve dónde hay saturación y a dónde se están yendo los turistas.

5.2 Lo que agregamos para la materia

- **Botón "A+" para la letra:** tres tamaños (Normal, Grande y Muy grande), visible desde la primera pantalla.
- **Formas en el semáforo:** cada estado tiene su forma (círculo, triángulo, cuadrado, rombo) además del color y el ícono. Así se entiende aunque no veas los colores.
- **Modo para daltonismo y alto contraste:** paletas alternativas que se activan con un botón.
- **Barra lateral ampliable:** un botón "Ver todos los lugares" la abre a casi toda la pantalla. Ahí aparecen los 23 lugares agrupados por color, sin scroll. Al elegir uno, la barra se cierra y muestra su ficha.

- **Filtros a la vista:** botones para filtrar por destino (Puebla, Cholula, Atlixco, Huejotzingo) y por interés.
- **Botones grandes:** todos de al menos 44 × 44 px.
- **Escuchar en voz alta:** la ficha del lugar y la recomendación se pueden escuchar, como en la práctica 10.
- **Pantalla de preferencias:** tamaño de letra, colores, contraste y voz en un solo lugar. Se recuerdan sin crear cuenta.
- **Guía de primer uso:** tres pasos que explican el semáforo, la barra lateral y el botón A+.

5.3 Con qué lo vamos a hacer: de React a PySide6

Todo el proyecto del hackathon está en React y Vite, pero en la materia trabajamos con **PySide6 y Qt Designer**. Para unir los dos mundos, separamos lo que se ve de lo que calcula:

- **La interfaz se rehace en PySide6:** cada pantalla se diseña en **Qt Designer** (archivos `.ui`) y se carga con `QUiLoader`, igual que en las prácticas. La navegación entre pantallas usa un `QStackedWidget`, y la barra lateral ampliable es un menú colapsable (`QFrame` con botones), como vimos en la Unidad 4.
- **El backend se queda igual:** la app de PySide6 le pide los datos al mismo servidor **FastAPI** del hackathon (semáforo, recomendaciones, reportes y asistente). Lo hace por HTTP, y el servidor responde en JSON. No hay que reescribir la lógica ni los modelos.
- **El mapa se reutiliza:** el mapa de Leaflet se muestra dentro de la ventana con `QWebEngineView` (ya viene en el PySide6 que tenemos instalado). Si conectar los clics del mapa con Qt se complica, el mapa queda solo para ver y la elección se hace desde la barra lateral.
- **Una versión "lite" con el mismo motor:** la app de PySide6 no copia toda la interfaz web. Es una versión más ligera, con menos pantallas y opciones, porque lo que vamos a hacer y mejorar es justamente la interfaz. Se apoya en el mismo backend, así que los datos y las recomendaciones son los mismos.
- **Las dos versiones conviven:** la página web de React se queda como la versión para celular que presentamos en el hackathon. La versión lite de PySide6 es la de la materia, pensada para computadora o para un **kiosco táctil** en módulos de información turística.

5.4 Cómo cumple el alcance mínimo

1. **Cuatro o más pantallas:** inicio, mapa con barra lateral, ficha del lugar, "Mejor ve a…", reporte, asistente y preferencias.
2. **Entrada, procesamiento y validación:**
   - **Entrada:** la persona elige sus gustos y su ubicación, reporta lugares y hace preguntas.
   - **Procesamiento:** el semáforo y las recomendaciones salen del backend.
   - **Validación:** no se puede empezar sin ubicación, solo puedes reportar un lugar cada 30 minutos y se avisa si no hay conexión.
3. **Tres decisiones de accesibilidad o más:** letra ajustable, formas y modo para daltonismo, barra lateral sin scroll, botones grandes y voz (detalle en Extras C).
4. **Pruebas con dos personas ajenas al equipo o más:** dos rondas, una con la versión del hackathon y otra con la nueva. Buscamos al menos un adulto mayor y, si se puede, una persona con daltonismo.
5. **Principios de IPM:** Gestalt, jerarquía visual, retroalimentación, consistencia y color (detalle en Extras D).
6. **Demostración reproducible:** instrucciones para instalar y correr el backend y la app. Los datos de afluencia son reales; si algo se simula en la demo, se indica en pantalla.

5.5 Cómo vamos a trabajar

Seguimos el ciclo del diseño centrado en el usuario que vimos en clase:

1. **Entender:** probamos la versión del hackathon con personas reales y anotamos dónde se atoran.
2. **Decidir:** de lo que encontremos salen los cambios.
3. **Diseñar:** armamos las pantallas en Qt Designer y las conectamos con el backend.
4. **Evaluar:** volvemos a probar con personas y aplicamos el cuestionario SUS. Lo que no funcione regresa al paso 3.

6. Nivel de complejidad previsto: Avanzado

Además de un flujo completo con validaciones y principios de IPM, el proyecto incluye funciones que van más allá de lo visto en el curso:

- un **asistente con inteligencia artificial** (un modelo de lenguaje local) que responde solo con los datos de la app;
- un **mapa en vivo** dentro de la ventana de Qt;
- datos de afluencia, tráfico y clima **en tiempo real**, consultados a un servidor;
- **lectura en voz alta** de la información.

Para ser claros: el backend y el asistente vienen del hackathon. Lo nuevo de este proyecto es la interfaz en PySide6, las funciones de accesibilidad y las pruebas con personas.

Extras

> **Material de apoyo**
> Estas secciones no forman parte del entregable 1, pero las conservamos porque servirán en los siguientes.

A. Diagnóstico de la versión del hackathon

Revisamos el código del frontend en React (`index.css` y componentes) y simulamos cómo se ven los colores del semáforo con daltonismo. Estos son los problemas que la versión en PySide6 debe evitar desde el diseño.

#	Hallazgo	Evidencia	A quién afecta
H1	**Texto pequeño.** El cuerpo mide 15 px y hay mucho texto de 11 a 13 px: horarios, traslados, insignias, créditos y fuentes del asistente	Escala tipográfica fija (13 / 15 / 17 / 22 / 28 px)	Adultos mayores, baja visión
H2	**El tamaño no responde a la configuración del usuario.** Todo está en píxeles fijos y no hay control para agrandar	Unidades `px` en toda la hoja de estilos	Quien necesita letra grande
H3	**Verde y rojo se confunden con daltonismo.** Con deuteranopía simulada, el verde y el rojo de los marcadores quedan con la misma luminosidad (1.00:1) y una diferencia de color ΔE ≈ 25, contra ΔE ≈ 108 en visión típica	Simulación de Machado (2009) sobre `#1F8A43` y `#C8281D`	Personas con daltonismo rojo-verde
H4	**Scroll para ver las opciones.** Los 23 lugares están en una barra de 400 px; se ven entre 5 y 8 a la vez (estimado a partir del CSS)	`.panel-flotante { width: 400px }`	Todos; en especial quien tiene poca práctica con la tecnología

#	Hallazgo	Evidencia	A quién afecta
H5	**Objetivos táctiles pequeños.** Selector de idioma de 30 px, sugerencias del asistente de ~30 px y botones de opción de 36 px	`.idioma button`, `.sugerencias button`	Motricidad fina reducida

Lo que ya estaba bien y se conserva: cada estado del semáforo lleva color, ícono y texto; el contraste del texto va de 6.0 a 7.2:1; hay foco visible, dos idiomas y la app se usa sin cuenta.

> **La trampa de cumplir la norma**
> La versión del hackathon ya cumplía varias reglas de WCAG y aun así no le funciona a todos. Cumplir la regla es necesario, pero no suficiente: lo que importa es que la persona logre su objetivo de forma fácil, clara y sin errores.

B. Detalle de las mejoras

Tamaño de letra.

- El tamaño base de la app se cambia en un solo lugar: la fuente de la aplicación y la hoja de estilos (QSS). Así, el botón A+ escala todo junto.
- El texto más pequeño será de al menos 14 px en tamaño Normal.
- Se revisa que en "Muy grande" (200 %) no quede texto cortado ni botones encimados.

Color y daltonismo.

- **Formas por estado, para todos:** el estado se reconoce aunque no se perciba ningún color.
- **Modo para daltonismo:** la paleta candidata es la de Okabe e Ito (azul, naranja y bermellón). En nuestra simulación, "buen momento" contra "mejor después" pasa de $\Delta E \approx 25$ a $\Delta E \approx 108$ con deuteranopía. "Con precaución" contra "mejor después" todavía queda cerca ($\Delta E \approx 18$), así que esos dos tonos se ajustan antes de adoptarla.
- **Alto contraste:** una hoja de estilos alterna con bordes y texto más oscuros, como el botón ◖ de la práctica 10.

Flujo de información.

- Barra lateral ampliable con los lugares agrupados en columnas por estado (Gestalt de proximidad y similitud).
- Filtros como botones visibles, no menús escondidos.
- El botón "← Todos los lugares" siempre en el mismo lugar.

Distintas edades y capacidades.

- Botones de al menos 44 × 44 px (`minimumSize` en Qt Designer), con separación entre ellos.
- Lenguaje simple: botones que dicen lo que hacen ("Vamos", "Ver todos los lugares"), sin términos técnicos.
- Uso completo con teclado: orden de tabulación definido en Qt Designer y foco siempre visible.
- Nombres accesibles (`accessibleName`) en los controles, para que el lector de pantalla los lea.
- Lectura en voz alta con `QTextToSpeech`.

C. Decisiones de accesibilidad justificadas

#	Decisión	Población	Fundamento	Cómo se verifica
D1	Botón A+ con tres tamaños de letra	Adultos mayores, baja visión	WCAG 1.4.4 (texto ampliable hasta 200 %)	En "Muy grande", nada se corta ni se encima
D2	Forma distinta por estado del semáforo	Daltonismo, baja visión	WCAG 1.4.1 (el color no es el único medio); Gestalt de similitud	Captura en escala de grises: cada estado se distingue
D3	Modo para daltonismo con paleta verificada	Daltonismo	WCAG 1.4.1 y 1.4.11 (contraste de componentes ≥ 3:1)	Simulación de deuteranopía, protanopía y tritanopía: $\Delta E \geq 40$ entre estados (umbral propuesto)
D4	Barra lateral ampliable, agrupada por estado	Poca experiencia digital, adultos mayores	Nielsen: reconocer antes que recordar; Gestalt de proximidad	Encontrar un lugar en buen momento toma menos tiempo y menos errores que en la prueba base
D5	Botones de 44 × 44 px o más	Motricidad reducida, kiosco táctil	WCAG 2.5.8 (mínimo 24 × 24 px); 44 px como meta	Revisión de todos los controles en Qt Designer
D6	Teclado, lector de pantalla y voz	Ceguera, baja visión, motricidad	WCAG 2.1.1 (teclado) y 2.4.7 (foco visible)	Recorrido completo solo con teclado y con el lector de pantalla Orca

D. Principios de IPM que aplica

Principio	Dónde se ve
Gestalt: proximidad y similitud	Lugares agrupados por estado; la misma forma y el mismo color para el mismo estado en el mapa, la lista y la ficha
Jerarquía visual	El nombre y el estado dominan; traslado y horario son secundarios
Retroalimentación	Confirmación al reportar; hora del último dato; aviso cuando el semáforo cambia

Principio	Dónde se ve
Consistencia	Mismos íconos, formas, colores y posición del botón de volver en todas las pantallas
Prevención de errores	"Estoy aquí" antes de reportar; el límite de reportes se explica; mensajes que dicen qué pasó y qué hacer
Reconocer antes que recordar	Ver todas las opciones a la vez en lugar de recorrer una lista
Color (regla 60–30–10)	Fondo neutro, rosa mexicano y azul talavera de marca, y el color fuerte reservado al semáforo
Inclusión	Preferencias de letra, color, contraste y voz; dos idiomas; sin cuenta

E. Uso de inteligencia artificial

En este documento (entregable 1):

- La idea, las mejoras y el enfoque son del equipo; los platicamos con la profesora antes de escribir.
- Usamos **Claude** (Claude Code, de Anthropic) para:
  - revisar el código de la versión del hackathon y sacar el diagnóstico de Extras A;
  - simular cómo se ven los colores del semáforo con daltonismo;
  - ordenar nuestras ideas y redactar el borrador.
- El borrador lo revisamos y corregimos nosotros (estructura, tono y alcance) antes de entregarlo.

En el desarrollo: usaremos asistentes de IA para investigar, depurar y apoyar la programación, como permite la guía del proyecto. Cada integrante debe poder explicar y modificar su parte del código sin IA, como se pedirá en la defensa.

En la solución: el asistente del turista usa un modelo de lenguaje local que responde solo con los datos de la app y cita sus fuentes.